

CS1010J Lecture #5

Debugging, etc.

Catch the bugs!



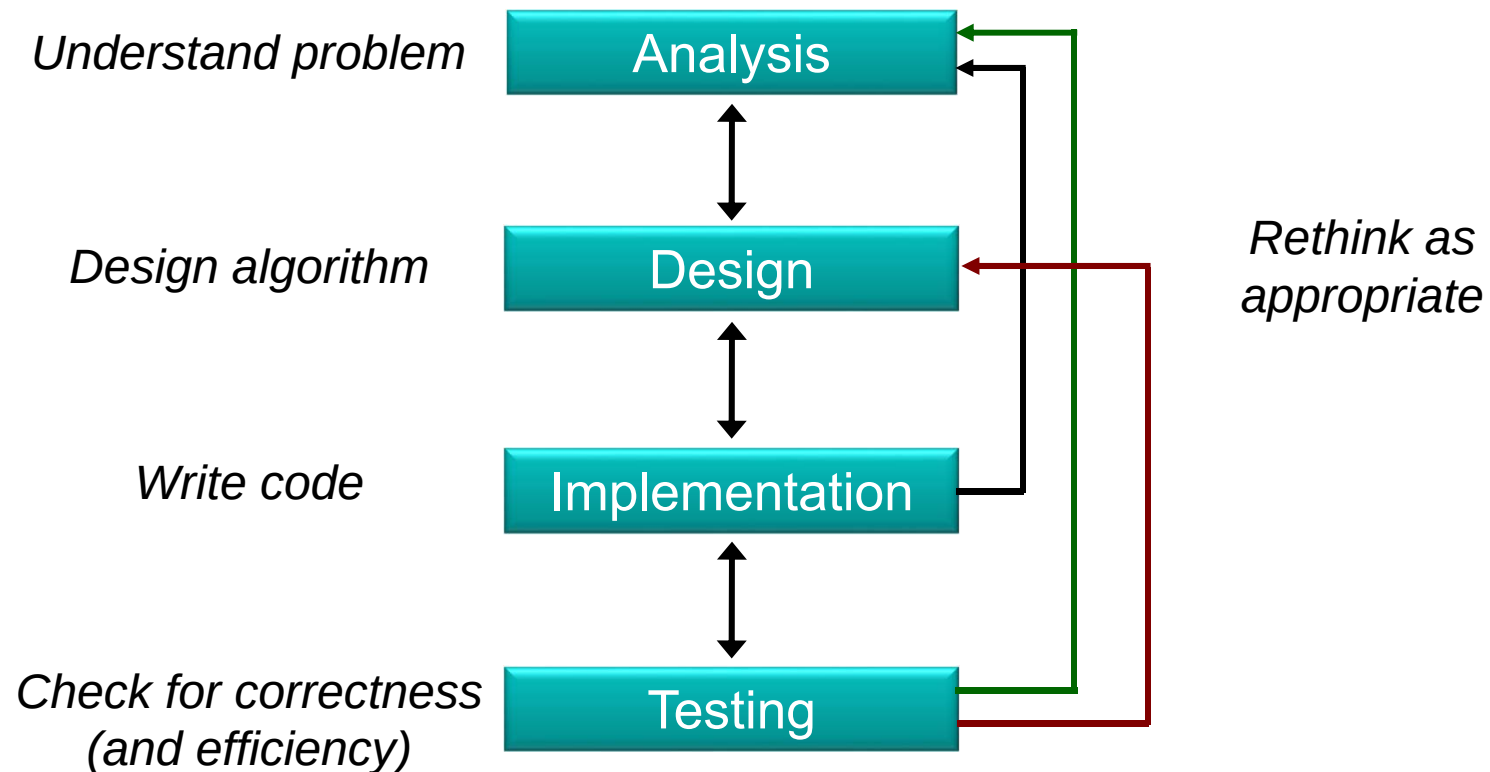
Department of Computer Science
School of Computing

Learning Objectives



- At the end of this lecture, you should understand:
 - ❑ What is testing and debugging.
 - ❑ How to strategically insert `println()` statements into a program for debugging purpose.
 - ❑ What are the commonly used problem solving strategies in CS1010J.

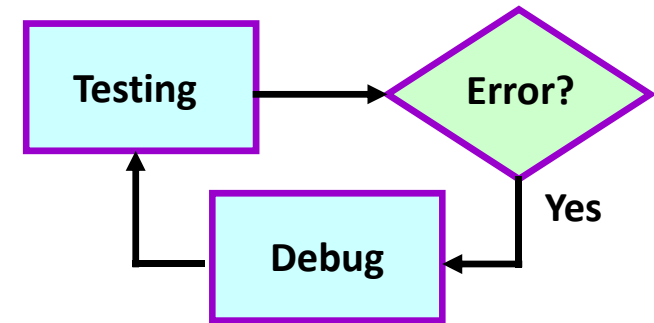
Problem Solving in Programming



Testing and Debugging

- Testing

- To check if a code contains errors.



- Debugging



- To locate the errors (“bugs”) and fix them.

- After debugging, the program is not necessarily error-free.

- It just means that whatever errors remain are harder to find. 😊
- This is especially true for large applications.

Programming Errors : Taxonomy

■ Syntax Error

- ❑ Reported by the compiler.
- ❑ Program **does not obey Java grammar** such as invalid expression, missing semi-colon, uninitialized variable, etc.

■ Run-time Error

- ❑ Reported at run-time.
- ❑ Program terminates unexpectedly (i.e. crashes) due to **illegal operation**, such as dividing a number by zero.
- ❑ Java's exception handling mechanism can catch such errors.

■ Logic Error

- ❑ Program finish execution but doesn't produce expected result (**wrong algorithm**).

Programming Errors : Examples

- Syntax errors

```
// code fragment from tutorial 2 Q5
double ratio = num1/num2;
if (ratio = 32.2) { // mistake 1
    System.out.println("Paper");
} else if (32.2 <= ratio < 45.8) { // mistake 2
    System.out.println("Pencil");
} else if (ratio < 90.2) {
    System.out.println("Ruler");
} else if (ratio < 100.0) {
    System.out.println("Eraser");
} else {
    System.out.println("Unknown");
}
```

Programming Errors : Examples

- Runtime errors

```
import java.util.Scanner;

class DemoRuntimeErrors {

    public static void main(String[] args) {

        System.out.print("Enter an integer: ");
        Scanner sc = new Scanner(System.in);

        int num = sc.nextInt();
        System.out.println(10 / num);
    }
}
```

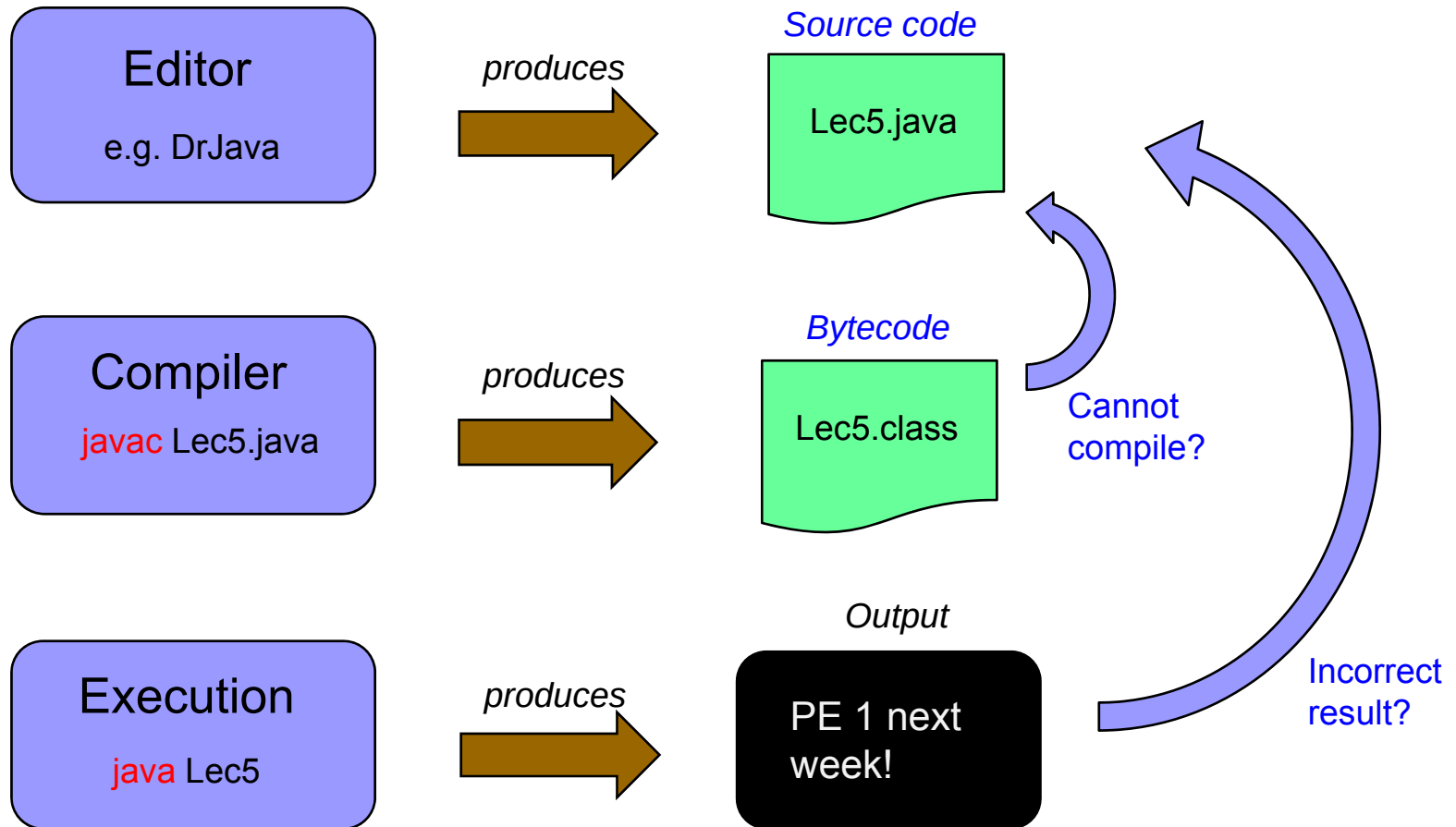
What if input is 0, or even 20.5, "abc"...

Programming Errors : Examples

- Logic error

```
x = 10
while (x > 5) {
    print(x)
}
```

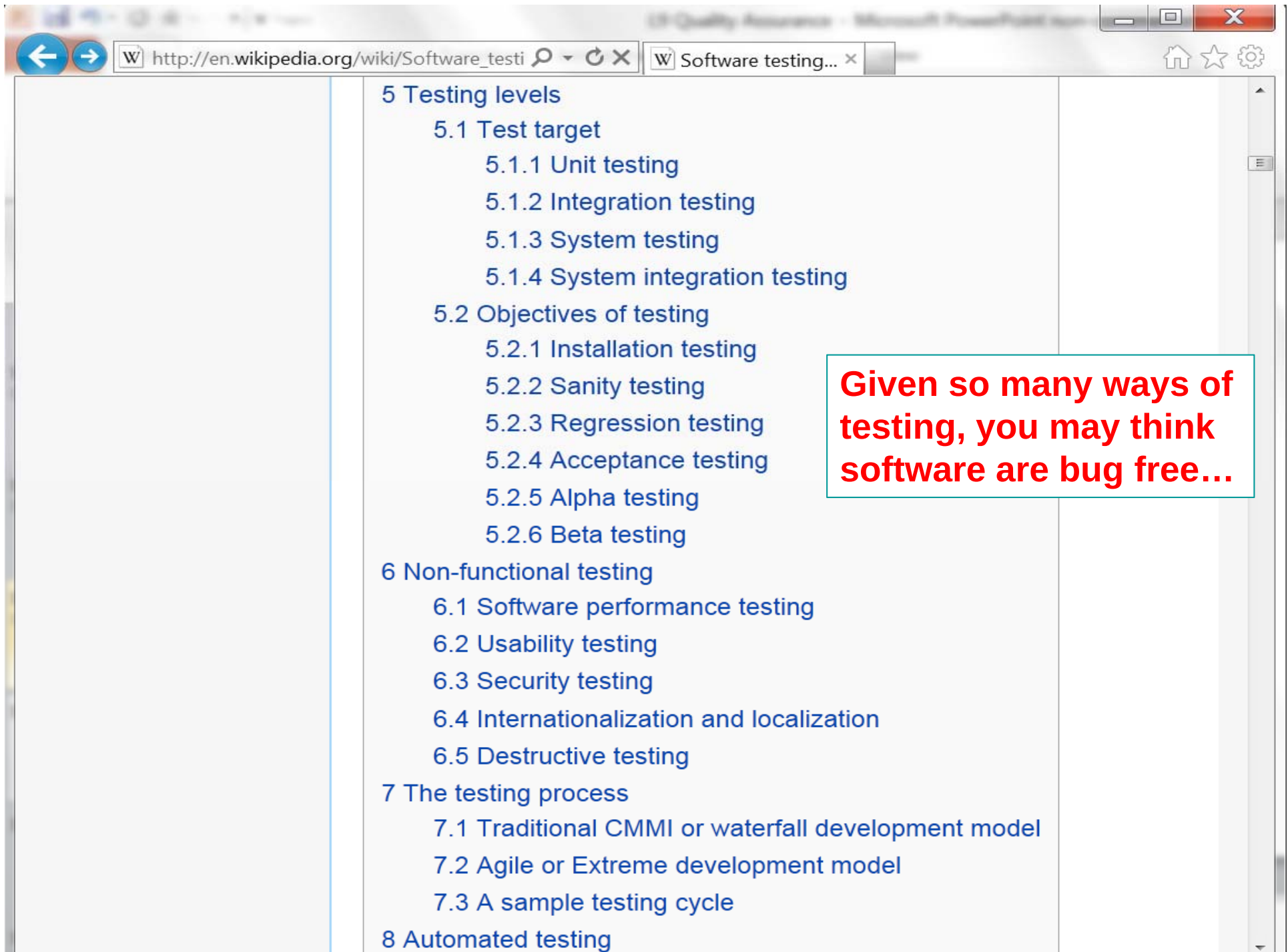

Testing Your Program



Taxonomy of Testing Methods

According to when testing is conducted:

- **Unit testing**
 - Test individual units (methods, classes, subsystems, etc.).
- **Integration testing**
 - Verify the correct interaction between subsystems.
- **System testing**
 - Test the system as a whole.
- **Regression testing**
 - Re-test everything after each small modification.
- **User acceptance testing**
 - Done by users to validate whether the delivered system meets user requirements.
- **More...**



5 Testing levels

- 5.1 Test target
 - 5.1.1 Unit testing
 - 5.1.2 Integration testing
 - 5.1.3 System testing
 - 5.1.4 System integration testing
- 5.2 Objectives of testing
 - 5.2.1 Installation testing
 - 5.2.2 Sanity testing
 - 5.2.3 Regression testing
 - 5.2.4 Acceptance testing
 - 5.2.5 Alpha testing
 - 5.2.6 Beta testing

6 Non-functional testing

- 6.1 Software performance testing
- 6.2 Usability testing
- 6.3 Security testing
- 6.4 Internationalization and localization
- 6.5 Destructive testing

7 The testing process

- 7.1 Traditional CMMI or waterfall development model
- 7.2 Agile or Extreme development model
- 7.3 A sample testing cycle

8 Automated testing

Given so many ways of testing, you may think software are bug free...

Windows

A fatal exception 0E has occurred at 0020:C0011E36 in UXD UMM(01) + 00010E36. The current application will be terminated.

- * Press any key to terminate the current application.
- * Press CTRL+ALT+DEL again to restart your computer. You will lose any unsaved information in all applications.

Press any key to continue



TTC



Infamous Programming Errors



■ Ariane 5 story (1996)

- ❑ **Cost:** \$370 million
- ❑ **Disaster:** Flight 501, which took place on June 4, 1996, was the first, and unsuccessful, test flight of the European Ariane 5 expendable launch system. As it was an unmanned flight, there were no victims, but the breakup caused the loss of four cluster mission spacecraft.
- ❑ **Cause:** Due to an error in the software design (**inadequate protection from integer overflow**), the rocket veered off its flight path 37 seconds after launch and was destroyed by its automated self-destruct system.

■ Mars Climate Crasher (1998)

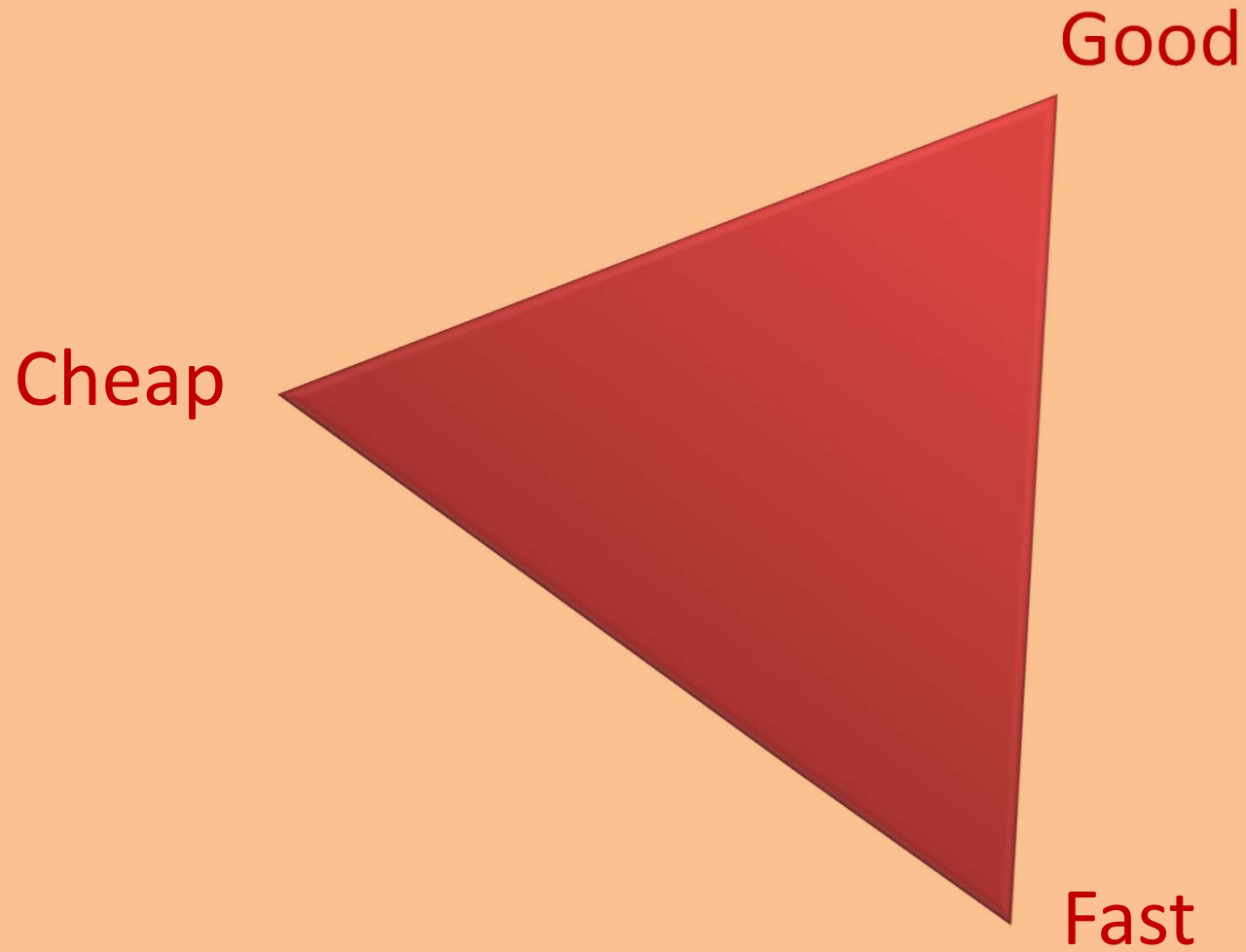


- ❑ **Cost:** \$125 million
- ❑ **Disaster:** After a 286-day journey from Earth, the Mars Climate Orbiter fired its engines to push into orbit around Mars. The engines fired, but the spacecraft fell too far into the planet's atmosphere, likely causing it to crash on Mars.
- ❑ **Cause:** The software that controlled the Orbiter thrusters **used imperial units (pounds of force), rather than metric units (Newtons)** as specified by NASA.

Difficulty of Software Testing

- The difficulty in software testing stems from the complexity of software: **we can not completely test a program of moderate complexity.**
- Software testing is a trade-off between **budget**, **time** and **quality**.

Good, cheap, fast: **select any two**



Testing & Debugging in Small-Scale (1/4)

- Philosophical notes on program design, debugging and testing
 - A good design is important
 - A good design results in a high quality program. A low quality design cannot be “debugged into” high quality.
 - Do not optimize for speed too early
 - It is possible to make a correct program run faster.
 - It is much more difficult to make a fast (but wrong) program run correctly.

Testing & Debugging in Small-Scale (2/4)

■ Tips:

- ❑ Start off with a working algorithm.
- ❑ Simplify your design and logic.
- ❑ Incremental coding, test early, fix bugs once you find them.
- ❑ Recognize common errors (e.g. integer division).
- ❑ Recompile the program for every single change.
- ❑ Test boundary values.
 - Example: in prime number test, did you test your program with boundary values 1 and 2?
- ❑ Explain the bug to someone else.
- ❑ Pause for a while and take a deep breath!

Testing & Debugging in Small-Scale (3/4)

- Think like a detective:
 - Look at the clues: error messages, variable values, etc.
 - Eliminate the impossible.
 - Run the program again with different inputs.
 - Does the same error occur again?

Testing & Debugging in Small-Scale (4/4)

- How to do debugging?
- Manual walkthroughs
 - Verbal walkthroughs.
 - Tracing with pencil-and-paper.
- Use print statements to check intermediate results
 - May provide information on:
 - Which methods have been called.
 - The value of parameters and local variables at strategic points

Example

```
...

// Extract the digits
int digit6 = number%10; number /= 10;
int digit5 = number%10; number /= 10;
int digit4 = number%10; number /= 10;
int digit3 = number%10; number /= 10;
int digit2 = number%10;

int step1 = digit2*2 + digit3*6 + digit4*2 + digit5*4 + digit6;
System.out.println("Step 1 = " + step1); // for checking

int step2 = step1 % 13;
System.out.println("Step 2 = " + step2); // for checking

int step3 = 13 - step2;
System.out.println("Step 3 = " + step3); // for checking

...
```

Using a Debugger

- Debugger usually provides the following
 - Stepping
 - Breakpoint
 - Watches (inspecting variables)
- DrJava comes with an embedded debugger. Explore the following documentation if interested to use it.

<http://drjava.org/docs/quickstart/ch04s03.html>

 - Note that installation of JDK (Java Development Kit) is required to use the debugger in DrJava. JDK can be downloaded from the Oracle Java website.

Problem Solving Methodology (1 / 3)

- Start from a hand example (e.g. sample run)
 - Get an idea how to do this example by yourself.
 - Note down all the major steps involved and their sequence.
 - Generalize your idea as necessary.
 - This is just your “algorithm”.

- This step requires good logical thinking ability
 - If you cannot work out the problem by hand, chances are that you won't be able to solve it on the computer either.
 - Practice and reflection should be helpful.

Problem Solving Methodology (2/3)

- Next, implement your algorithm into a program.
 - Translate each step into a program fragment (e.g. a method).
 - If a step is quite complex, break it down further into several smaller program fragments (e.g. several simpler methods).
 - This is known as “step-wise refinement” or “top-down” design.
 - Imagine the structure of your program/method before typing.
 - Major *data structure* and *control structure* needed?
- This step requires your familiarity with syntax of a programming language and knowledge of common programming strategies.
 - A lot of practice is the best way out.
 - Digest the programs of others.

Problem Solving Methodology (3/3)

- Finally, verify that your program works correctly.
 - Train yourself of the following good habit.
 - When finish coding, press **Ctrl+A** then **Tab** to auto-indent your program.
 - Browse through your program one more time.
 - Any messy indentation indicate the existence of syntax errors (e.g. missing semicolon, unmatched brackets, etc.).
 - For debugging purpose, strategically insert **print** statements into your program to print out intermediate value.
 - Check the correctness of intermediate values to narrow down the scope of checking
 - Test every branch of your logic.

Q1

(Simplified from CS1010 AY2016/17 Semester 1 Midterm Test, Q10)

- One way to calculate the constant pi (π) is given by the formula as shown below, where each fraction (without the sign) is considered a term.

$$\pi = \frac{4}{1} - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \dots$$

- Write a program that reads in an integer *noTerms*, which represents the number of terms to be used for calculation, prints the value of π , corrected to 2 decimal places.

Enter number of terms: 5
The value of pi is 3.34

Q2

(CS1010 AY2012/13 Semester 1 Midterm Test, Q8)

- Write a method called `printPyramid()` to print the pyramid of numbers as shown below.
 - You are to use loops, and not hardcode using 5 `print()` statements.

```
1*2*3*4*5*6*7*8*9*
3*4*5*6*7*8*9*
5*6*7*8*9*
7*8*9*
9*
```

Homework

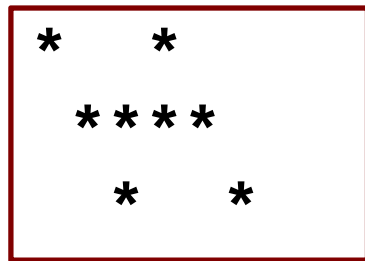
- Write a method called `printPyramidV2()` to print the pyramid of numbers as shown below.
 - You are to use loop, and not hardcode using 5 `print()` statements.

```
1*  
1*2*3*  
1*2*3*4*5*  
1*2*3*4*5*6*7*  
1*2*3*4*5*6*7*8*9*
```

Homework

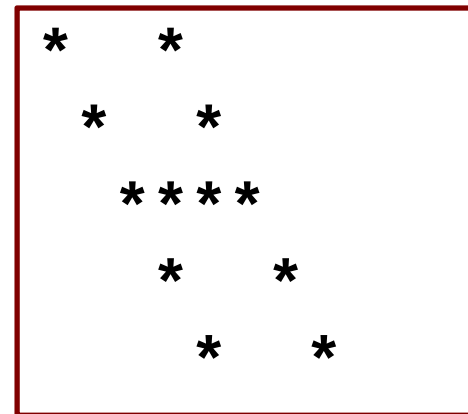
(CS1010 AY2010/11 Semester 1 Midterm Test, Q16)

- Write a method `pattern_H()` that takes in a positive odd integer parameter n larger than 1, and outputs a slanted **H** pattern, made up of "`*`", with n rows.



A 3x3 grid of asterisks forming a slanted 'H' shape. The first and third rows have asterisks at the first and third columns. The middle row has asterisks at the second, third, and fourth columns.

$n = 3$



A 5x5 grid of asterisks forming a slanted 'H' shape. The first and fifth rows have asterisks at the first and fifth columns. The middle three rows have asterisks at the second, third, fourth, and fifth columns.

$n = 5$

Simple Exception Handling

**Non-
examinable**

- Report error elegantly when data of wrong type is entered by the user.
 - `try ... catch`

```
try {  
    System.out.print("Enter an integer: ");  
    num = sc.nextInt();  
} catch (InputMismatchException ime) {  
    System.err.println("Only integer allowed.");  
} // end try-catch
```

Today's Summary

Debugging; Problem Solving

Testing and Debugging

- Manually trace the program for checking.
- Strategically insert print statement to print out intermediate result for manual checking.

Common problem solving strategies

- Divide-and-conquer
 - Start from an example, design a solution and generalize it
 - Deal with common cases first; special cases later
-
- Java Exception Handling

